

LAPORAN MILESTONE 4 TUGAS BESAR
IF2224 TEORI BAHASA FORMAL DAN AUTOMATA

Intermediate Code dan Interpreter



Disusun oleh:

JBC - J1B2C2

Samuelson D. Tanuraharja	13524001
Aufa Rienaldifaza Ahmad	13524027
Niko Samuel Simanjuntak	13524029
Aurelia Jennifer Gunawan	13524089
Reinsen Silitonga	13524093

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132

2026

DAFTAR ISI

DAFTAR ISI	2
DAFTAR GAMBAR	2
DAFTAR TABEL	4
BAB 1	
TEORI SINGKAT	5
1.1. Arsitektur Compiler dan Interpreter	5
1.2. Intermediate Code	5
1.3. Arsitektur Mesin Eksekusi	5
1.4. Runtime Error dan Keamanan Eksekusi	5
BAB 2	
PERANCANGAN DAN IMPLEMENTASI	6
2.1. Teknologi yang Digunakan	6
2.2. Struktur Program	6
2.3. Fungsi/Kelas Utama dan Implementasinya	7
2.4.1. File ASTNode.hpp dan ASTNode.cpp	7
2.4.2. File ASTBuilder.hpp dan ASTBuilder.cpp	12
2.4.3. File SymbolTable.hpp dan SymbolTable.cpp	13
2.4.4. File SemanticAnalyzer.hpp dan SemanticAnalyzer.cpp	14
2.4.5. File visit_declarations.cpp	14
2.4.6. File visit_statements.cpp	15
2.4.7. File visit_expressions.cpp	15
2.4.8. File TypeChecker.hpp dan TypeChecker.cpp	16
2.4.9. File DecoratedASTPrinter.hpp dan DecoratedASTPrinter.cpp	16
2.4.10. File main.cpp	16
2.4. Alur Kerja Program	17
2.5. Justifikasi Implementasi	18
BAB 3	
PENGUJIAN	19
BAB 4	
KESIMPULAN DAN SARAN	20
4.1. Kesimpulan	20
4.2. Saran	20
BAB 5	
LAMPIRAN	21
5.1. Link Release Repository GitHub	21
5.2. Pembagian Tugas	21
5.3. Referensi	21

DAFTAR GAMBAR

Gambar 1. Diagram alur eksekusi program	6
Gambar 2. Stack Machine	11
Gambar 3. Register Machine	11
Gambar 4. Struktur Direktori Program Intermediate Code Generator dan Interpreter	14

DAFTAR TABEL

Tabel 2.4.1. Implementasi kelas ICGenerator	14
Tabel 2.4.2. Implementasi kelas ICInstruction	18
Tabel 2.4.3. Implementasi kelas ICGen_Expr	18
Tabel 2.4.4. Implementasi kelas header ICGen_Statements	20
Tabel 2.4.5. Implementasi kelas ICGen_Subprograms	23
Tabel 2.4.8. Implementasi kelas StackMachine	25
Tabel 2.4.9. Implementasi kelas Interpreter	26
Tabel 2.4.10. Implementasi kelas main	29
Tabel 3.1. Pengujian program	33
Tabel 5.3. Pembagian tugas	40

BAB 1

TEORI SINGKAT

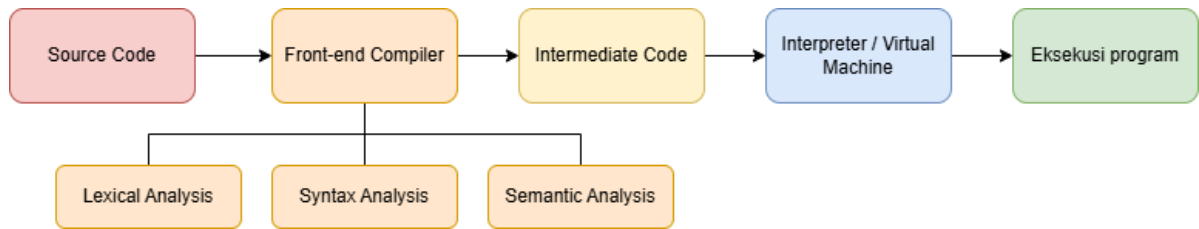
1.1. Arsitektur Compiler dan Interpreter

Compiler merupakan perangkat lunak yang bertugas menerjemahkan program yang ditulis dalam *high-level language* ke dalam bentuk lebih rendah yang dapat dipahami dan dijalankan oleh komputer. Proses kompilasi umumnya terdiri atas serangkaian tahap yang saling berhubungan, mulai dari analisis program hingga pembangkitan representasi yang siap dieksekusi. Dalam implementasi compiler modern, proses tersebut tidak selalu dilakukan langsung dari *source code* menuju *machine code*, melainkan melalui representasi perantara yang memungkinkan pemisahan antara tahap analisis dan tahap eksekusi.

Arsitektur *compiler* dapat dibagi menjadi dua bagian utama, yaitu bagian *front-end* dan bagian *back-end*. Bagian *front-end* bertanggung jawab melakukan analisis terhadap program sumber untuk memahami struktur dan maknanya. Tahap ini mencakup *lexical analysis*, *syntax analysis*, dan *semantic analysis*. Hasil dari proses tersebut adalah representasi program yang dapat diterjemahkan ke bentuk yang lebih dekat dengan mekanisme eksekusi. Selanjutnya, bagian *back-end* bertugas mengubah hasil analisis tersebut menjadi representasi yang dapat dijalankan oleh sistem. Mayoritas *compiler modern* menggunakan representasi *Intermediate Code* atau *Intermediate Representation (IR)*. *Intermediate code* berperan sebagai lapisan abstraksi yang memisahkan proses analisis program dari proses eksekusi. Dengan adanya lapisan ini, *compiler* menjadi lebih modular karena perubahan pada bahasa sumber maupun platform target tidak selalu memengaruhi keseluruhan sistem.

Dalam sistem yang menggunakan *interpreter*, *intermediate code* tidak diterjemahkan lebih lanjut menjadi *machine code*, tetapi dieksekusi secara langsung oleh sebuah mesin abstrak atau *virtual machine*. *Interpreter* bertugas membaca, menafsirkan, dan menjalankan instruksi pada *intermediate code*. Pendekatan ini memungkinkan satu representasi program digunakan pada berbagai platform selama tersedia *interpreter* yang memahami *instruction set* yang digunakan.

Secara umum, alur eksekusi program dapat digambarkan sebagai berikut



Gambar 1. Diagram alur eksekusi program

Melalui pendekatan tersebut, proses translasi program menjadi lebih fleksibel, mudah dikembangkan, dan tidak bergantung secara langsung pada karakteristik perangkat keras tertentu.

1.2. Intermediate Code

Intermediate Code atau *Intermediate Representation* (IR) merupakan representasi program yang digunakan sebagai perantara antara bahasa sumber dan mekanisme eksekusi. Dalam arsitektur *compiler* modern, IR berfungsi sebagai lapisan abstraksi yang memisahkan proses analisis program dari proses pembangkitan kode maupun eksekusi program. Dengan adanya lapisan ini, compiler tidak perlu menerjemahkan setiap konstruksi bahasa tingkat tinggi secara langsung ke instruksi mesin tertentu. *Intermediate representation* yang baik memenuhi beberapa karakteristik penting antara lain sebagai berikut:

- representasi harus mudah dihasilkan dari struktur program yang telah dianalisis
- representasi harus cukup sederhana sehingga mudah dianalisis, dimodifikasi, atau dioptimasi oleh compiler
- representasi harus cukup dekat dengan model eksekusi yang digunakan sehingga dapat diterjemahkan atau dieksekusi secara efisien

Berdasarkan bentuk representasinya, *Intermediate Representation* dapat berupa struktur pohon, grafik, atau urutan instruksi linear. *Abstract Syntax Tree* (AST), *Three-Address Code* (TAC), *Static Single Assignment* (SSA), dan *stack-based intermediate code* merupakan beberapa bentuk IR yang banyak digunakan dalam implementasi *compiler* modern. Masing-masing bentuk memiliki karakteristik dan tujuan penggunaan yang berbeda sesuai kebutuhan sistem yang dibangun.

1.2.1. *Three-Address Code (TAC)*

Three-Address Code (TAC) adalah salah satu bentuk *intermediate representation* yang umum digunakan dalam *compiler*. Pada *Three-Address Code*, setiap instruksi melibatkan paling banyak tiga alamat, yaitu dua operand dan satu lokasi penyimpanan hasil. Pendekatan ini bertujuan untuk memecah ekspresi kompleks menjadi serangkaian operasi sederhana yang lebih mudah dianalisis dan diproses oleh *compiler*. Sebagai contoh, ekspresi $a := b + c * d$ dapat direpresentasikan dalam TAC sebagai berikut

```
t1 := c * d  
t2 := b + t1  
a := t2
```

Variabel sementara digunakan untuk menyimpan hasil perhitungan antara sebelum digunakan oleh operasi berikutnya. Dengan demikian, setiap instruksi hanya melakukan satu operasi sederhana sehingga dependensi antaroperasi dapat diamati dengan lebih jelas.

Keunggulan *Three-Address Code (TAC)* adalah kemampuannya dalam menyederhanakan proses analisis dan optimasi program. *Compiler* dapat dengan mudah melakukan berbagai transformasi, seperti *constant folding*, *dead code elimination*, atau analisis aliran data, karena setiap operasi direpresentasikan secara eksplisit. Selain itu, TAC juga menjadi bentuk representasi yang relatif mudah diterjemahkan ke berbagai arsitektur target. Pada beberapa sistem, TAC hanya digunakan sebagai representasi konseptual sebelum diterjemahkan ke bentuk lain yang lebih dekat dengan mesin eksekusi.

1.2.2. *Translasi Ekspresi ke Intermediate Code*

Generator intermediate code bertugas dalam menerjemahkan ekspresi yang terdapat pada program menjadi rangkaian instruksi yang dapat dieksekusi oleh mesin abstrak. Proses ini dilakukan dengan mengubah operasi-operasi tingkat tinggi menjadi operasi dasar yang lebih sederhana dan terstruktur. Translasi umumnya dilakukan melalui traversal terhadap representasi sintaksis program. Pada setiap node ekspresi, *generator* kode menghasilkan instruksi yang sesuai dengan operator dan operand yang terlibat. Untuk ekspresi biner seperti penjumlahan, pengurangan, perkalian, atau

pembagian, operand terlebih dahulu dievaluasi sebelum operasi dilakukan. Pendekatan ini memastikan bahwa urutan evaluasi tetap konsisten dengan aturan bahasa pemrograman. Sebagai contoh, pernyataan $x := a + b$ dapat diterjemahkan menjadi urutan instruksi yang memuat nilai a dan b , melakukan operasi penjumlahan, kemudian menyimpan hasilnya ke variabel x . Pada implementasi berbasis stack machine, proses tersebut dapat direpresentasikan sebagai berikut:

LOD a
LOD b
OPR ADD
STO x

Selain ekspresi aritmetika, proses translasi juga mencakup ekspresi logika, assignment statement, dan operasi unary. Dengan memecah ekspresi menjadi operasi-operasi elementer, compiler dapat menghasilkan representasi yang lebih sederhana dan mudah dieksekusi oleh interpreter.

1.2.3. *Translasi Control Flow*

Berbeda dengan ekspresi aritmetika yang umumnya diterjemahkan secara linear, struktur *control flow* memerlukan mekanisme untuk mengubah urutan eksekusi instruksi. *Control flow* menentukan bagaimana program berpindah antar bagian berdasarkan kondisi tertentu atau struktur perulangan yang digunakan. *Compiler* menggunakan konsep *label* dan *jump instruction* untuk merepresentasikan *control flow*. Label berfungsi sebagai penanda lokasi tertentu dalam urutan instruksi, sedangkan jump instruction digunakan untuk mengubah alur eksekusi menuju lokasi yang ditentukan. Melalui mekanisme ini, konstruksi tingkat tinggi seperti IF, IF-ELSE, dan WHILE dapat direpresentasikan menggunakan instruksi yang relatif sederhana. Pada percabangan IF, *compiler* menghasilkan instruksi evaluasi kondisi yang diikuti oleh *conditional jump* menuju bagian setelah blok IF apabila kondisi bernilai salah.

Pada percabangan IF-ELSE, *conditional jump* digunakan untuk memilih blok yang akan dieksekusi, sedangkan *unconditional jump* digunakan untuk melewati blok yang tidak dipilih. Sementara itu, pada perulangan WHILE, *compiler* menghasilkan instruksi yang memungkinkan eksekusi kembali ke awal blok selama kondisi masih bernilai benar.

1.2.4. *Instruction Set pada Intermediate Code*

Instruction set merupakan sekumpulan instruksi yang dapat dipahami dan dieksekusi oleh suatu mesin abstrak atau *virtual machine*. *Instruction set* berfungsi sebagai antarmuka antara hasil translasi program dan mekanisme eksekusi yang digunakan oleh *interpreter*. *Instruction set* dapat dikelompokkan menjadi beberapa kategori. Instruksi manipulasi data digunakan untuk memuat atau menyimpan nilai dari dan ke memori. Instruksi aritmetika dan logika digunakan untuk melakukan operasi perhitungan atau evaluasi kondisi. Instruksi pengendalian alur digunakan untuk mengatur perpindahan eksekusi program, sedangkan instruksi manajemen prosedur digunakan untuk mendukung pemanggilan dan pengembalian prosedur. Penggunaan *instruction set* memungkinkan seluruh konstruksi bahasa pemrograman direpresentasikan menggunakan sejumlah operasi dasar yang terbatas.

Pada implementasi compiler Arion, *intermediate code* direpresentasikan menggunakan *instruction set* berbasis *stack machine*. Instruksi INT digunakan untuk mengalokasikan ruang memori yang diperlukan selama eksekusi program. Instruksi LIT digunakan untuk memasukkan nilai literal ke dalam stack. Instruksi LOD digunakan untuk memuat nilai dari memori ke stack,. Instruksi STO digunakan untuk menyimpan nilai dari *stack* ke memori. Instruksi OPR merepresentasikan operasi aritmetika dan logika, yang memungkinkan *interpreter* melakukan berbagai operasi terhadap nilai yang berada pada *stack*. Instruksi JMP dan JPC digunakan untuk pengendalian alur program. Instruksi RET digunakan untuk mengakhiri eksekusi atau mengembalikan kontrol program ke konteks sebelumnya. Kombinasi instruksi-instruksi tersebut membentuk bahasa perantara yang digunakan sebagai dasar komunikasi antara *compiler* dan *interpreter*.

1.3. Arsitektur Mesin Eksekusi

1.3.1. *Virtual Machine dan Interpreter*

Virtual machine merupakan mesin abstrak yang dirancang untuk mengeksekusi suatu *instruction set* tanpa bergantung pada perangkat keras. Tidak seperti prosesor nyata yang mengeksekusi *machine code*, *virtual machine* menjalankan instruksi yang didefinisikan oleh sistem. Pendekatan ini memungkinkan perancang *compiler* untuk

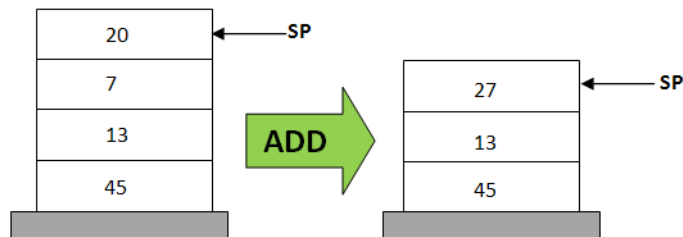
menentukan model eksekusi yang sesuai dengan kebutuhan aplikasi tanpa harus terikat pada arsitektur perangkat keras.

Interpreter merupakan komponen yang mengimplementasikan *virtual machine*. *Interpreter* membaca instruksi satu per satu, menentukan operasi yang harus dilakukan, kemudian menjalankannya sesuai definisi *instruction set* yang digunakan. Dengan demikian, *interpreter* bertindak sebagai lapisan yang menjembatani *intermediate code* dengan sumber daya komputasi yang tersedia pada sistem. Penggunaan *virtual machine* meningkatkan portabilitas program, dan menyederhanakan implementasi *compiler* karena proses translasi hanya perlu menghasilkan instruksi untuk mesin abstrak yang telah didefinisikan sebelumnya.

1.3.2. Siklus *Fetch-Decode-Execute*

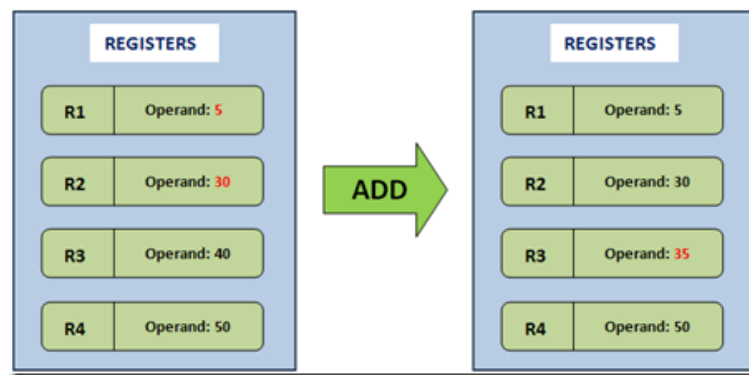
Eksekusi instruksi pada interpreter umumnya mengikuti siklus *fetch-decode-execute*. Siklus ini merupakan mekanisme dasar yang menentukan bagaimana setiap instruksi diproses hingga menghasilkan perubahan pada keadaan sistem. Pada tahap *fetch*, interpreter mengambil instruksi dari lokasi memori yang ditunjuk oleh *Program Counter* (PC). Selanjutnya, pada tahap *decode*, *interpreter* mengidentifikasi jenis instruksi serta operand yang diperlukan. Setelah instruksi berhasil diinterpretasikan, tahap *execute* dilakukan untuk menjalankan operasi yang bersesuaian. Setelah eksekusi selesai, nilai *Program Counter* diperbarui sehingga *interpreter* dapat melanjutkan ke instruksi berikutnya. Apabila instruksi yang dijalankan merupakan instruksi lompatan, nilai *Program Counter* dapat berubah menuju lokasi lain yang ditentukan oleh instruksi tersebut. Siklus ini berlangsung secara berulang kali hingga program mencapai kondisi berhenti.

1.3.3. *Stack Machine*



Gambar 2. Stack Machine

Stack machine merupakan arsitektur mesin eksekusi yang menggunakan struktur *stack* sebagai media utama penyimpanan operand dan hasil operasi. *Stack machine* melakukan sebagian besar operasinya melalui mekanisme *push* dan *pop* terhadap *stack*. Pada *register machine*, instruksi biasanya harus menyebutkan lokasi operand secara eksplisit. Misalnya, operasi penjumlahan dapat direpresentasikan sebagai ADD R1, R2, R3 yang berarti nilai pada register R2 dan R3 dijumlahkan dan hasilnya disimpan pada R1. Sementara itu, pada *stack machine*, operand selalu diambil dari puncak *stack* sehingga instruksi menjadi lebih sederhana, yaitu PUSH 5 PUSH 3 ADD. Instruksi ADD secara otomatis mengambil dua nilai teratas dari *stack*, melakukan operasi penjumlahan, kemudian mengembalikan hasilnya ke *stack*.



Gambar 3. Register Machine

1.3.4. *Stack Frame dan Manajemen Memori*

Interpreter memerlukan mekanisme pengelolaan memori yang terstruktur untuk mendukung proses eksekusi program. Salah satu mekanisme yang umum digunakan adalah *stack frame* atau *activation record*. *Stack frame* menyimpan seluruh informasi yang diperlukan selama eksekusi suatu blok program atau prosedur. Setiap kali eksekusi memasuki prosedur baru, sebuah *stack frame* baru dibuat dan ditempatkan di atas *stack*. Ketika prosedur selesai dijalankan, *stack frame* tersebut dihapus dan kontrol program dikembalikan ke *caller*. Mekanisme ini memungkinkan beberapa prosedur aktif dikelola secara sistematis tanpa saling mengganggu.

Komponen yang umumnya terdapat dalam *stack frame* meliputi *static link*, *dynamic link*, *return address*, dan area penyimpanan variabel lokal. *Static link*

digunakan untuk mengakses lingkungan leksikal yang lebih luar, *dynamic link* digunakan untuk melacak *stack frame* pemanggil, sedangkan *return address* menyimpan lokasi instruksi yang harus dieksekusi setelah prosedur selesai dijalankan.

1.3.5. Eksekusi Instruksi Intermediate Code

Eksekusi *intermediate code* dilakukan dengan menjalankan instruksi secara berurutan sesuai mekanisme *fetch-decode-execute*. Setiap instruksi yang dijalankan dapat memengaruhi isi *stack*, memori, dan alur kontrol program. Instruksi seperti LIT dan LOD digunakan untuk menambahkan nilai ke *stack*. Selanjutnya, instruksi OPR mengambil operand yang diperlukan dari puncak *stack*, melakukan operasi yang diminta, dan mengembalikan hasilnya ke *stack*. Instruksi STO kemudian dapat digunakan untuk menyimpan hasil tersebut ke lokasi memori yang sesuai.

Interpreter menggunakan instruksi JMP dan JPC untuk mengendalikan alur program. Instruksi JMP menyebabkan perpindahan eksekusi tanpa syarat menuju lokasi tertentu, sedangkan JPC melakukan perpindahan hanya jika kondisi yang dievaluasi bernilai salah.

BAB 2

PERANCANGAN DAN IMPLEMENTASI

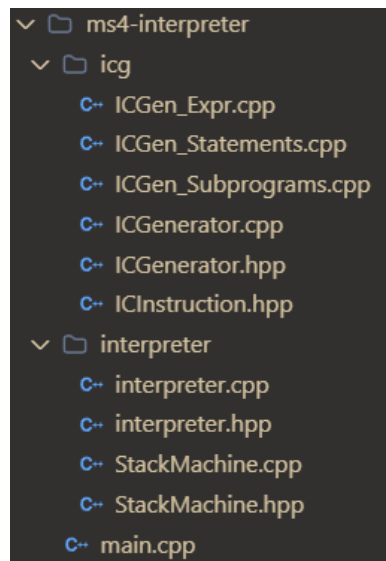
2.1. Teknologi yang Digunakan

Program *intermediate code generator* dan *interpreter* ditulis dengan bahasa pemrograman C++ (standar C++11 atau lebih tinggi) tanpa dependensi dari pihak ketiga. Pemilihan C++ didasarkan pada kebutuhan akan performa tinggi dalam pemrosesan token dan fleksibilitas dalam pengelolaan memori secara eksplisit. Lingkungan pengembangan yang digunakan bersifat portabel, dengan dukungan *build tool* CMake versi minimal 3.16 untuk menjamin kompilasi lintas *platform*. Program bekerja dalam empat tahap, yaitu tokenisasi, *parsing*, konstruksi AST, beserta analisis semantik, pembangkitan *intermediate code*, dan eksekusi melalui *interpreter*. Tokenisasi dilakukan oleh *lexical analyzer* yang telah dikembangkan pada milestone 1. Sedangkan *parsing* dengan pendekatan *recursive descent* yang telah dikembangkan pada milestone 2 menghasilkan *parse tree* sebagai representasi awal struktur program. Lalu, *parse tree* dikonversi menjadi *abstract syntax tree* (AST) oleh modul ASTBuilder dengan struktur *node* untuk deklarasi, *statement*, ekspresi, dan tipe data. Analisis semantik dilakukan oleh *semantic analyzer*, yang diimplementasi pada milestone 3 menggunakan pola *Visitor* yang menghasilkan *decorated AST* sebagai input untuk *code generator*. Kemudian, pada milestone ini, dikembangkan *generator intermediate code* yang melakukan traversal *Abstract Syntax Tree* secara *depth-first* untuk menghasilkan instruksi *intermediate code* berbasis *stack machine*. Instruksi yang dihasilkan kemudian dieksekusi oleh *interpreter* yang memanfaatkan *stack machine* sebagai mesin abstrak.

2.2. Struktur Program

Arsitektur program *intermediate code generator* dan *interpreter* dirancang secara modular dengan pemisahan tanggung jawab yang jelas antar komponen. Komponen utama terdiri dari *ICInstruction*, *ICGenerator*, *StackMachine*, dan *Interpreter*. Struct *ICInstruction* merepresentasikan satu instruksi *intermediate code* dengan atribut nomor baris, nama operasi, argumen pertama, dan argumen kedua. Kelas *ICGenerator* menjadi komponen sentral yang mengorkestrasikan proses pembangkitan *intermediate code* melalui traversal *Abstract Syntax Tree* dengan pola *visitor*. Implementasinya terbagi ke dalam tiga *file* sesuai dengan kategori node, yaitu *ICGen_Expr.cpp* untuk menangani *expression* dan *assignment*, *ICGen_Statements.cpp* untuk menangani struktur kontrol, dan *ICGen_Subprograms.cpp* untuk menangani deklarasi prosedur

dan fungsi. Kelas `ICGenerator` berinteraksi dengan `SymbolTable` yang dihasilkan pada tahap *semantic analysis* untuk memperoleh informasi pada level *lexical* dan alamat *runtime* setiap simbol. Informasi tersebut digunakan untuk menghasilkan instruksi yang sesuai melalui perhitungan selisih level *lexical* dan *offset* memori *runtime*. Daftar instruksi yang dihasilkan kemudian dieksekusi oleh Interpreter menggunakan `StackMachine` sebagai representasi memori *runtime* berbasis *stack* yang mendukung pengelolaan *activation record*, *static link*, dan akses variabel lintas cakupan sesuai aturan *lexical scoping* hingga program selesai dijalankan atau terjadi *runtime error*.



Gambar 4. Struktur Direktori Program *Intermediate Code Generator* dan *Interpreter*

2.3.Fungsi/Kelas Utama dan Implementasinya

2.4.1. File *ICGenerator.hpp* dan *ICGenerator.cpp*

Tabel 2.4.1. Implementasi kelas `ICGenerator`

Kelas/Fungsi/Prosedur	Penjelasan
<code>ICGenerator(ms3::SymbolTable& symTable)</code> <pre>ICGenerator::ICGenerator(ms3::SymbolTable& symTable) : symTable_(symTable) {}</pre>	Konstruktor yang menerima referensi ke <code>SymbolTable</code> hasil analisis semantik. Menyimpan referensi tersebut ke <code>symTable_</code> untuk digunakan saat menghitung alamat dan level runtime variabel.

<pre>vector<ICInstruction> generate(ASTNode* root) std::vector<ICInstruction> ICGenerator::generate(ASTNode* root) { code_.clear(); nextLine_ = 0; visitNode(root); return code_; }</pre>	Entry point utama generator. Mereset state internal (code_, nextLine_), kemudian memulai traversal AST dari node root dengan memanggil visitNode(). Mengembalikan vector berisi seluruh instruksi IC yang dihasilkan.
<pre>void writeToFile(const vector<ICInstruction>& code, const string& filename) void ICGenerator::writeToFile(const std::vector<ICInstruction>& code, const std::string& filename) { std::ofstream out(filename); if (!out) throw std::runtime_error("Cannot open output file: " + filename); for (const auto& instr : code) { out << instr.line << " " << instr.op << " " << instr.level << " " << instr.arg << "\n"; } }</pre>	Menulis daftar instruksi IC ke file teks. Setiap instruksi ditulis dalam format: nomor_baris op level arg, dipisahkan spasi per baris.
<pre>void emit(const string& op, int level, int arg) // Tambahkan satu instruksi IC ke daftar dan naikan nomor baris berikutnya void ICGenerator::emit(const std::string& op, int level, int arg) { code_.push_back(ICInstruction(nextLine++, op, level, arg)); }</pre>	Menambahkan satu instruksi IC baru ke vector code_ dengan nomor baris saat ini (nextLine_), lalu menaikkan nextLine_.
<pre>void backpatch(int targetLine, int value) void ICGenerator::backpatch(int targetLine, int value) { code_[targetLine].arg = value; }</pre>	Mengisi ulang nilai arg dari instruksi di indeks targetLine dengan value. Digunakan untuk melengkapi alamat lompatan JMP/JPC yang belum diketahui saat instruksi pertama kali di-emit.
<pre>void visitNode(ASTNode* node) // Teruskan node ke visitor yang sesuai berdasarkan jenis node-nya void ICGenerator::visitNode(ASTNode* node) { if (!node) return; switch (node->kind) { case ASTKind::Program: visitProgram(static_cast<ProgramNode*>(node)); break; case ASTKind::Block: visitBlock(static_cast<BlockNode*>(node)); break; case ASTKind::Assign: visitAssign(static_cast<AssignNode*>(node)); break; case ASTKind::BinOp: visitBinOp(static_cast<BinOpNode*>(node)); break; case ASTKind::UnaryOp: visitUnaryOp(static_cast<UnaryOpNode*>(node)); break; case ASTKind::Var: visitVar(static_cast<VarNode*>(node)); break; case ASTKind::ArrayAccess: visitArrayAccess(static_cast<ArrayAccessNode*>(node)); break; case ASTKind::RecordAccess: visitRecordAccess(static_cast<RecordAccessNode*>(node)); break; case ASTKind::IntLit: visitIntLit(static_cast<IntLitNode*>(node)); break; case ASTKind::RealLit: visitRealLit(static_cast<RealLitNode*>(node)); break; case ASTKind::CharLit: visitCharLit(static_cast<CharLitNode*>(node)); break; case ASTKind::StringLit: visitStringLit(static_cast<StringLitNode*>(node)); break; case ASTKind::BoolLit: visitBoolLit(static_cast<BoolLitNode*>(node)); break; case ASTKind::FuncCall: visitFuncCall(static_cast<FuncCallNode*>(node)); break; case ASTKind::Compound: visitCompound(static_cast<CompoundNode*>(node)); break; case ASTKind::IF: visitIf(static_cast<IfNode*>(node)); break; case ASTKind::While: visitWhile(static_cast<WhileNode*>(node)); break; case ASTKind::For: visitFor(static_cast<ForNode*>(node)); break; case ASTKind::Repeat: visitRepeat(static_cast<RepeatNode*>(node)); break; case ASTKind::Case: visitCase(static_cast<CaseNode*>(node)); break; case ASTKind::ProcCall: visitProcCall(static_cast<ProcCallNode*>(node)); break; case ASTKind::ProcDecl: visitProcDecl(static_cast<ProcDeclNode*>(node)); break; case ASTKind::FuncDecl: visitFuncDecl(static_cast<FuncDeclNode*>(node)); break; // VarDecl, ConstDecl, TypeDecl tidak perlu emit IC karena sudah ditangani M3 default: break; } }</pre>	Dispatcher utama yang meneruskan node ke visitor yang sesuai berdasarkan field node->kind menggunakan switch-case. Node bertipe deklarasi variabel, konstanta, dan tipe tidak memerlukan emit instruksi IC sehingga diabaikan di sini.

<pre> void visitProgram(ProgramNode* n) // Emit frame program utama, Lewati subprogram dengan JMP, lalu jalankan body void ICGenerator::visitProgram(ProgramNode* n) { int varCount = 0; for (ASTNode* decl : n->declarations) { if (decl->kind == ASTKind::VarDecl) { auto* vd = static_cast<VarDeclNode*>(decl); for (const auto& name : vd->names) { int idx = lookupInBlock(name, 1); if (idx != -1) { ms3::TabEntry& entry = symTable_.getTabEntry(idx); varCount += symTable_.typeSize(entry.type, entry.ref); } else { varCount += 1; } } } } emit("INT", 0, 3 + varCount); bool hasSubprogram = false; for (ASTNode* decl : n->declarations) { if (decl->kind == ASTKind::ProcDecl decl->kind == ASTKind::FuncDecl) { hasSubprogram = true; break; } } int jmpLine = -1; if (hasSubprogram) { jmpLine = currentLine(); emit("JMP", 0, 0); } for (ASTNode* decl : n->declarations) { if (decl->kind == ASTKind::ProcDecl decl->kind == ASTKind::FuncDecl) { visitNode(decl); } } if (hasSubprogram) { backpatch(jmpLine, currentLine()); } visitNode(n->body); emit("RET", 0, 0); } </pre>	Mengemit instruksi INT untuk mengalokasikan frame program utama (3 slot header + jumlah variabel global). Jika ada subprogram, mengemit JMP untuk melewati kode subprogram ke body utama, memanggil visitor subprogram, lalu backpatch alamat JMP tersebut. Mengakhiri program dengan RET.
<pre> void visitBlock(BlockNode* n) // Emit deklarasi non-variabel dan body untuk blok subprogram void ICGenerator::visitBlock(BlockNode* n) { bool hasSubprogram = false; for (ASTNode* decl : n->declarations) { if (decl->kind == ASTKind::ProcDecl decl->kind == ASTKind::FuncDecl) { hasSubprogram = true; break; } } int jmpLine = -1; if (hasSubprogram) { jmpLine = currentLine(); emit("JMP", 0, 0); } for (ASTNode* decl : n->declarations) { if (decl->kind != ASTKind::VarDecl && decl->kind != ASTKind::ConstDecl && decl->kind != ASTKind::TypeDecl) { visitNode(decl); } } if (hasSubprogram) { backpatch(jmpLine, currentLine()); } visitNode(n->body); } </pre>	Menangani blok milik subprogram: melewati kode subprogram bersarang (jika ada) dengan JMP, memanggil visitor deklarasi non-variabel, backpatch JMP, lalu mengunjungi body blok.
<pre> int runtimeLevel(const TabEntry& entry) int ICGenerator::runtimeLevel(const ms3::TabEntry& entry) const { int diff = currentLevel_ - entry.lev; return diff < 0 ? 0 : diff; } </pre>	Menghitung perbedaan lexical level antara scope saat ini (currentLevel_) dan level deklarasi variabel. Nilai ini digunakan sebagai argumen level pada instruksi LOD/STO untuk traversal static link.

<pre>int runtimeAddr(const TabEntry& entry) { int ICGenerator::runtimeAddr(const ms3::TabEntry& entry) const { if (!frameStack_.empty() && entry.lev == currentLevel_) { const FrameInfo& f = frameStack_.back(); if (f.isFunction) { return 4 + entry.adr; } } return 3 + entry.adr; } }</pre>	Menghitung offset absolut variabel di dalam frame stack-nya. Fungsi mendapatkan offset +4 (ada return slot) atau +3 (tidak ada return slot) dari base pointer, bergantung pada apakah frame adalah milik fungsi atau prosedur/program utama.
<pre>int lookupInBlock(const string& name, int btabIdx) { int ICGenerator::lookupInBlock(const std::string& name, int btabIdx) const { if (btabIdx < 0 btabIdx >= symTable_.btabsSize()) return -1; int idx = symTable_.getTabEntry(btabIdx).last; while (idx >= 1) { const auto& entry = symTable_.getTabEntry(idx); if (entry.name == name) { return idx; } idx = entry.link; } return -1; } }</pre>	Menelusuri linked list simbol dalam btabs untuk mencari entri dengan nama tertentu. Mengembalikan indeks entri di tab jika ditemukan, atau -1 jika tidak ada.
<pre>int countParams(const std::vector<ParamNode*>& params) const { int ICGenerator::countParams(const std::vector<ParamNode*>& params) const { int count = 0; for (ParamNode* p : params) { if (p) count += static_cast<int>(p->names.size()); } return count; } }</pre>	Menghitung total jumlah parameter dari daftar ParamNode dengan menjumlahkan ukuran vector names pada setiap ParamNode. Hasilnya digunakan oleh visitProcDecl dan visitFuncDecl untuk menentukan ukuran frame yang perlu dialokasikan.
<pre>int countLocalVars(BlockNode* block) const { int ICGenerator::countLocalVars(BlockNode* block) const { int count = 0; if (!block) return 0; for (ASTNode* decl : block->declarations) { if (decl && decl->kind == ASTKind::VarDecl) { count += static_cast<int>(static_cast<VarDeclNode*>(decl)->names.size()); } } return count; } }</pre>	Menghitung jumlah variabel lokal dalam sebuah blok dengan mengiterasi daftar deklarasi dan menghitung total names pada setiap VarDeclNode. Digunakan bersama countParams() untuk menentukan total slot frame yang dibutuhkan saat menghasilkan instruksi INT.
<pre>int currentLine() const { // Kembalikan nomor baris yang akan dipakai oleh emit berikutnya int ICGenerator::currentLine() const { return nextLine_; } }</pre>	Mengembalikan nilai nextLine_ saat ini, yaitu nomor baris instruksi IC berikutnya yang akan di-emit. Digunakan oleh visitWhile, visitRepeat, dan visitFor untuk mencatat alamat awal kondisi loop sebelum instruksi ditulis, sehingga JMP di akhir loop dapat melompat kembali ke alamat yang tepat.

2.4.2. File *ICInstruction.hpp*

Tabel 2.4.2. Implementasi kelas ICInstruction

Kelas/Fungsi/Prosedur	Penjelasan
<pre>struct ICInstruction { int line; std::string op; int level; int arg; }</pre>	Struktur data yang merepresentasikan satu instruksi intermediate code. Menyimpan empat field: line (nomor urut instruksi), op (nama operasi, misalnya "LIT", "LOD", "OPR"), level (argumen pertama yang biasanya menyatakan lexical level atau jumlah parameter), dan arg (argumen kedua berupa nilai literal, alamat, atau kode operasi OPR).
<pre>ICInstruction(int line, const string& op, int level, int arg) : line(line), op(op), level(level), arg(arg) {}</pre>	Konstruktor yang menginisialisasi keempat field sekaligus. Digunakan oleh ICGenerator::emit() setiap kali satu instruksi baru dihasilkan.

2.4.3. File *ICGen_Expr.cpp*

Tabel 2.4.3. Implementasi kelas ICGen_Expr

Kelas/Fungsi/Prosedur	Penjelasan
<pre>void visitIntLit(IntLitNode* n) { void ICGenerator::visitIntLit(IntLitNode* n) { emit("LIT", 0, n->value); } }</pre>	Menghasilkan instruksi LIT 0 <value> untuk literal integer.
<pre>void visitRealLit(RealLitNode* n) { void ICGenerator::visitRealLit(RealLitNode* n) { emit("LIT", 0, static_cast<int>(n->value)); } }</pre>	Menghasilkan instruksi LIT 0 <value> untuk literal real; nilai di-truncate ke integer karena stack machine berbasis integer.
<pre>void visitCharLit(CharLitNode* n) { void ICGenerator::visitCharLit(CharLitNode* n) { emit("LIT", 0, static_cast<int>(n->value)); } }</pre>	Menghasilkan instruksi LIT 0 <ascii> untuk literal karakter; nilai karakter dikonversi ke representasi ASCII-nya.

<pre>void visitBoolLit(BoolLitNode* n) void ICGenerator::visitBoolLit(BoolLitNode* n) { emit("LIT", 0, n->value ? 1 : 0); }</pre>	Menghasilkan instruksi LIT 0 1 untuk true dan LIT 0 0 untuk false.
<pre>void visitStringLit(StringLitNode* n) void ICGenerator::visitStringLit(StringLitNode* n) { (void)n; emit("LIT", 0, 0); }</pre>	Stub placeholder; menghasilkan LIT 0 0 tanpa memperhitungkan konten string. Dukungan string literal penuh di luar scope M4.
<pre>void visitVar(VarNode* n) void ICGenerator::visitVar(VarNode* n) { if (n->sem.tab_index < 0) { throw std::runtime_error("ICGen Error: VarNode '" + n->name + "' memiliki tab_index tidak valid -- tidak dianotasi oleh SemanticAnalyzer"); } ms3::TabEntry& entry = symTable_.getTabEntry(n->sem.tab_index); emit("LOD", runtimeLevel(entry), runtimeAddr(entry)); }</pre>	Menghasilkan instruksi LOD untuk membaca nilai variabel dari frame stack. Address disesuaikan dengan offset +3 untuk melewati frame header (sl, dl, ra), dan level dihitung secara relatif terhadap kedalaman nesting saat ini.
<pre>void visitAssign(AssignNode* n) void ICGenerator::visitAssign(AssignNode* n) { if (n->target->kind == ASTKind::ArrayAccess) { emitAddr(n->target); visitNode(n->value); emit("STO", 0, 0); } else if (n->target->kind == ASTKind::Var) { visitNode(n->value); auto* varNode = static_cast<VarNode*>(n->target); if (varNode->sem.tab_index < 0) { throw std::runtime_error("ICGen Error: assignment target '" + varNode->name + "' memiliki tab_index tidak valid yang tidak dianotasi oleh SemanticAnalyzer"); } ms3::TabEntry& entry = symTable_.getTabEntry(varNode->sem.tab_index); if (entry.opd == OBJ_FUNCTION) { emit("STO", 0, 3); } else { emit("STO", runtimeLevel(entry), runtimeAddr(entry)); } } }</pre>	Menghasilkan instruksi STO untuk menyimpan hasil ekspresi ke variabel target. Mendeteksi kasus OBJ_FUNCTION (konvensi return value Pascal) di mana nilai RHS dibiarkan di eval stack tanpa di-STO.
<pre>void visitBinOp(BinOpNode* n) void ICGenerator::visitBinOp(BinOpNode* n) { visitNode(n->left); visitNode(n->right); static const std::unordered_map<std::string, int> opMap = { {"+", 2}, // ADD {"-", 3}, // SUB {"*", 4}, // MUL {"/", 5}, // DIV (real division) {"div", 5}, // DIV (integer division) {"mod", 6}, // MOD {"=", 7}, // EQL (Arion menggunakan '=' untuk perbandingan) {"==", 7}, // EQL (alternatif) {"<>", 8}, // NEQ {"<", 9}, // LSS {">=", 10}, // GEQ {">", 11}, // GTR {"<=", 12}, // LEQ {"and", 4}, // AND via MUL: benar untuk boolean (0*x=0, 1*1=1) }; if (n->op == "or") { emit("OPR", 0, 2); // ADD emit("LIT", 0, 0); emit("OPR", 0, 8); // NEQ: 1 jika (sum != 0), 0 jika (sum == 0) return; } auto it = opMap.find(n->op); if (it != opMap.end()) { emit("OPR", 0, it->second); } }</pre>	Menghasilkan instruksi OPR untuk operasi biner: aritmatika (+, -, *, div, mod), perbandingan (=, <, <=, >, <=, >=), dan logika (and, or). Operator or ditangani secara khusus dengan normalisasi ADD + NEQ untuk memastikan hasil selalu 0 atau 1.

<pre>void visitUnaryOp(UnaryOpNode* n) void ICGenerator::visitUnaryOp(UnaryOpNode* n) { visitNode(n->operand); if (n->op == "-") { emit("OPR", 0, 1); } else if (n->op == "not") { emit("LIT", 0, 0); emit("OPR", 0, 7); } }</pre>	Menghasilkan instruksi OPR 1 (negasi aritmatika) untuk operator -, dan LIT 0 + OPR EQL untuk operator not.
<pre>void visitFuncCall(FuncCallNode* n) void ICGenerator::visitFuncCall(FuncCallNode* n) { for (ASTNode* arg : n->args) { visitNode(arg); } auto it = funcLabels_.find(n->name); if (it != funcLabels_.end()) { emit("CAL", -static_cast<int>(n->args.size()) - 1, it->second); } else { throw std::runtime_error("ICGen Error: fungsi '" + n->name + "' tidak ditemukan di funcLabels_"); } }</pre>	Mengevaluasi dan mem-push seluruh argumen ke eval stack, lalu menghasilkan instruksi CAL ke label fungsi yang dituju berdasarkan funcLabels_.
<pre>void visitArrayAccess(ArrayAccessNode* n) void ICGenerator::visitArrayAccess(ArrayAccessNode* n) { emitAddr(n); emit("LODI", 0, 0); }</pre>	Menghasilkan instruksi alamat array via emitAddr (emit LODA + IXA) lalu LODI untuk membaca nilai dari alamat tersebut. Instruksi IXA di interpreter menyertakan validasi batas indeks (kaitannya dengan bonus Out-of-Bounds).
<pre>void visitRecordAccess(RecordAccessNode* n) void ICGenerator::visitRecordAccess(RecordAccessNode* n) { (void)n; throw std::runtime_error("ICGen Error: akses field record belum diimplementasikan (fitur bonus M4)"); }</pre>	Melempar runtime_error karena akses field record tidak diimplementasikan; mencegah stack corruption yang terjadi jika tidak ada nilai yang dipush ke eval stack.
<pre>void emitAddr(ASTNode* node) void ICGenerator::emitAddr(ASTNode* node) { if (node->kind == ASTKind::Var) { auto* varNode = static_cast<VarNode*>(node); if (varNode->sym.tab_index < 0) { throw std::runtime_error("ICGen Error: VarNode '" + varNode->name + "' has invalid tab_index"); } m3::TabEntry* entry = symTable_.getTabEntry(varNode->sym.tab_index); emit("LODA", runtimeLevel.entry, runtimeAddr(entry)); } else if (node->kind == ASTKind::ArrayAccess) { auto* arrNode = static_cast<ArrayAccessNode*>(node); emitAddr(arrNode->array); visitNode(arrNode->index); int atabIdx = arrNode->array->sym.ref; m3::AtaEntry* a = symTable_.getAtaEntry(atabIdx); emit("IXA", a.low, a.high); } else { throw std::runtime_error("ICGen Error: L-value not supported for address generation"); } }</pre>	Menghasilkan alamat l-value dari sebuah node tanpa membaca nilainya. Untuk VarNode mengemit LODA, untuk ArrayAccessNode memanggil emitAddr rekursif pada array-nya lalu mengemit IXA dengan batas indeks dari AtaEntry.

2.4.4. File ICGen_Statements.cpp

Tabel 2.4.4. Implementasi kelas header ICGen_Statements

Kelas/Fungsi/Prosedur	Penjelasan
-----------------------	------------

<pre>void visitCompound(CompoundNode* n) void ICGenerator::visitCompound(CompoundNode* n) { if (!n) return; for (ASTNode* stmt : n->statements) { visitNode(stmt); } }</pre>	Melakukan traversal berurutan pada kumpulan <i>statement</i> di dalam blok program (begin, end). Prosedur ini memanggil visitNode secara rekursif untuk setiap sub-<i>statement</i> di dalamnya.
<pre>void visitIf(IfNode* n) void ICGenerator::visitIf(IfNode* n) { if (!n) return; visitNode(n->condition); int jpcLine = currentLine(); emit("JPC", 0, 0); visitNode(n->thenBranch); if (n->elseBranch) { int jmpLine = currentLine(); emit("JMP", 0, 0); backpatch(jpcLine, currentLine()); visitNode(n->elseBranch); backpatch(jmpLine, currentLine()); } else { backpatch(jpcLine, currentLine()); } }</pre>	Mengompilasi evaluasi kondisi pecabangan if. Prosedur ini membangkitkan instruksi JPC untuk melompat melewati blok then jika kondisi bernilai <i>false</i>. Jika ada blok percabangan else, instruksi JMP ditambahkan pada akhir blok then agar eksekusi melompat keluar.
<pre>void visitWhile(WhileNode* n) void ICGenerator::visitWhile(WhileNode* n) { if (!n) return; int startLine = currentLine(); visitNode(n->condition); int jpcLine = currentLine(); emit("JPC", 0, 0); visitNode(n->body); emit("JMP", 0, startLine); backpatch(jpcLine, currentLine()); }</pre>	Mengelola instruksi untuk perulangan while. Pengecekan dilakukan menggunakan instruksi JPC untuk keluar dari struktur while jika kondisi bernilai false. Di baris terakhir <i>body</i> iterasi, dibangkitkan instruksi JMP untuk melakukan pengecekan ulang ke baris kondisi.
<pre>void visitFor(ForNode* n)</pre>	Mengelola instruksi untuk perulangan for. Awalnya, nilai fromExpr di simpan ke <i>stack frame</i> menggunakan STO. Batas iterasi dievaluasi dengan perbandingan $\geq$ (OPR 10) untuk alur <i>downto</i> atau $\leq$ (OPR 12) untuk alur <i>to</i> lalu diikuti JPC untuk menyelesaikan kondisi keluar. Di ujung <i>body</i>, variabel pengontrol ditambahkan atau dikurangi sebesar LIT 1

<pre> void ICGenerator::visitFor(ForNode* n) { if (!n) return; int idx = -1; for (int i = symTable_.tabSize() - 1; i >= 1; --i) { if (symTable_.getTabEntry(i).name == n->varName) { idx = i; break; } } if (idx == -1) { throw std::runtime_error("Runtime Error: Loop variable not found: " + n->varName); } TabEntry& entry = symTable_.getTabEntry(idx); // Evaluate starting value and store into variable visitNode(n->fromExpr); emit("STO", runtimeLevel(entry), runtimeAddr(entry)); int condStart = currentLine(); // Evaluate current value of loop variable emit("LOD", runtimeLevel(entry), runtimeAddr(entry)); // Evaluate boundary expression visitNode(n->toExpr); // Compare if (n->downto) { emit("OPR", 0, 10); // GEQ (>=) } else { emit("OPR", 0, 12); // LEQ (<=) } int jmpLine = currentLine(); emit("JPC", 0, 0); // Body visitNode(n->body); // Step (increment / decrement loop variable) emit("LOD", runtimeLevel(entry), runtimeAddr(entry)); emit("LIR", 0, 1); if (n->downto) { emit("OPR", 0, 3); // SUB } else { emit("OPR", 0, 2); // ADD } emit("STO", runtimeLevel(entry), runtimeAddr(entry)); emit("JMP", 0, condStart); backpatch jmpLine, currentLine(); } </pre>	sebelum melompat balik menggunakan JMP.
void visitRepeat(RepeatNode* n) <pre> void ICGenerator::visitRepeat(RepeatNode* n) { if (!n) return; int startLine = currentLine(); for (ASTNode* stmt : n->body) { visitNode(stmt); } visitNode(n->condition); // Repeat runs until condition is true, i.e., jumps if condition is false emit("JPC", 0, startLine); } </pre>	Mengelola instruksi untuk perulangan repeat-until. Mengeksekusi blok kumpulan <i>statement-list</i> yang ada di dalam <i>body</i> perulangan terlebih dahulu. Jika nilai kondisi bernilai false, lompat balik ke baris awal deklarasi <i>body</i> menggunakan instruksi JPC.
void visitCase(CaseNode* n)	Mengimplementasikan struktur <i>pattern matching</i> pada blok case. Nilai ekspresi dibandingkan secara sekuensial dengan blok <i>case label</i> menggunakan instruksi EQL. Karena JPC, melompat saat nilai berupa nol, disisipkan logika negasi sebelum pemanggilan JPC. Instruksi percabangan diakhiri dengan JMP yang dikumpulkan dalam <i>backpath</i> agar eksekusi melompat keluar setelah salah <i>case</i> terpenuhi.

<pre> void ICGenerator::visitCase(CaseNode* n) { if (!n) return; std::vector<int> endJmps; for (CaseBranchNode* branch : n->branches) { std::vector<int> labelJpcs; for (ASTNode* label : branch->labels) { visitNode(n->expr); visitNode(label); emit("OPR", 0, 7); // EQL // Negate condition so JPC jumps to statement if true emit("LIT", 0, 0); emit("OPR", 0, 7); // EQL to 0 (NOT) labelJpcs.push_back(currentLine()); emit("JPC", 0, 0); } int skipBranchJmp = currentLine(); emit("JMP", 0, 0); int stmtStart = currentLine(); for (int jmpLine : labelJpcs) { backpatch(jmpLine, stmtStart); } visitNode(branch->statement); endJmps.push_back(currentLine()); emit("JMP", 0, 0); backpatch(skipBranchJmp, currentLine()); } int endLine = currentLine(); for (int jmpLine : endJmps) { backpatch(jmpLine, endLine); } } </pre>	
void visitProcCall(ProcCallNode* n) <pre> void ICGenerator::visitProcCall(ProcCallNode* n) { if (!n) return; std::string lowerName = n->name; std::transform(lowerName.begin(), lowerName.end(), lowerName.begin(), [](unsigned char c) { return std::tolower(c); }); if (lowerName == "writeln" lowerName == "write") { if (lowerName == "write") { for (ASTNode* arg : n->args) { visitNode(arg); emit("OPR", 0, 13); // int: print top of stack without newline } } else { if (n->args.empty()) { throw std::runtime_error("Runtime Error: Writeln requires at least one argument"); } for (size_t i = 0; i < n->args.size(); i++) { visitNode(n->args[i]); emit("OPR", 0, 1 + i == n->args.size() ? 14 : 13); } } } else { for (ASTNode* arg : n->args) { visitNode(arg); } auto it = funcLabels.find(n->name); if (it != funcLabels.end()) { emit("CAL", static_cast<int>(n->args.size()), it->second); } } } </pre>	Mengelola instruksi untuk pemanggilan prosedur. Jika <i>identifer</i> terdeteksi sebagai fungsi bawaan, yaitu <i>writeln</i> dan <i>write</i>, masing-masing dilakukan pembangkitan OPR 13 dan OPR 14. Untuk prosedur <i>user-defined</i>, dibangkitkan instruksi CAL yang merujuk pada pemetaan memori di <i>funcLabels_</i>.

2.4.5. File ICGen_Subprograms.cpp

Tabel 2.4.5. Implementasi kelas ICGen_Subprograms

Kelas/Fungsi/Prosedur	Penjelasan
-----------------------	------------

<pre> void visitProcDecl(ProcDeclNode* n) { void ICGenerator::visitProcDecl(ProcDeclNode* n) { if (!n) return; funcLabels_[n->name] = currentLine(); int paramCount = 0; for (ParamNode* param : n->params) { paramCount += static_cast<int>(param->names.size()); } int varCount = 0; if (n->block) { for (ASTNode* decl : n->block->declarations) { if (decl->kind == ASTKind::VarDecl) { auto* vd = static_cast<VarDeclNode*>(decl); for (const auto& name : vd->names) { int idx = lookupInBlock(name, n->sem.ref); if (idx != -1) { ms3::TabEntry& entry = symTable_.getTabEntry(idx); varCount += symTable_.typeSize(entry.type, entry.ref); } else { varCount += 1; } } } } } emit("INT", paramCount, 3 + paramCount + varCount); FrameInfo f; f.paramCount = paramCount; f.isFunction = false; f.functionName = n->name; frameStack_.push_back(f); currentLevel++; currentDepth++; // masuk ke scope prosedur visitNode(n->block); currentDepth--; // keluar dari scope prosedur emit("RET", 0, 0); frameStack_.pop_back(); currentLevel--; } } </pre>	Mengelola instruksi untuk deklarasi prosedur. Prosedur ini mencatat alamat baris awal prosedur, menghitung jumlah parameter, dan menghitung memori variabel lokal dengan mempertimbangkan tipe data dinamis. Prosedur ini menginstruksikan INT dengan alokasi sebesar <i>header frame</i> (3) + jumlah parameter + variabel lokal. Setelah menelusuri blok internal prosedur, dibangkitkan instruksi RET 0 0 untuk menandakan kembalinya eksekusi ke pemanggil tanpa nilai kembalian.
<pre> void visitFuncDecl(VisitFuncDecl* n) { void ICGenerator::visitFuncDecl(FuncDeclNode* n) { if (!n) return; funcLabels_[n->name] = currentLine(); int paramCount = 0; for (ParamNode* param : n->params) { paramCount += static_cast<int>(param->names.size()); } int varCount = 0; if (n->block) { for (ASTNode* decl : n->block->declarations) { if (decl->kind == ASTKind::VarDecl) { auto* vd = static_cast<VarDeclNode*>(decl); for (const auto& name : vd->names) { int idx = lookupInBlock(name, n->sem.ref); if (idx != -1) { ms3::TabEntry& entry = symTable_.getTabEntry(idx); varCount += symTable_.typeSize(entry.type, entry.ref); } else { varCount += 1; } } } } } emit("INT", paramCount, 4 + paramCount + varCount); FrameInfo f; f.paramCount = paramCount; f.isFunction = true; f.functionName = n->name; frameStack_.push_back(f); currentLevel++; currentDepth++; // masuk ke scope fungsi visitNode(n->block); currentDepth--; // keluar dari scope fungsi emit("RET", 0, 1); frameStack_.pop_back(); currentLevel--; } } </pre>	Mengelola instruksi untuk deklarasi fungsi. Logika perhitungan pembangkitan instruksi INT mirip seperti prosedur, namun digunakan 4 slot <i>header</i>. Penambahan 1 slot ini dialokasikan khusus sebagai tempat penyimpanan nilai kembalian fungsi. Setelah menelusuri blok internal fungsi, dibangkitkan instruksi RET 0 1 untuk menandakan kembalinya eksekusi ke pemanggil dengan nilai kembalian.

2.4.6. File StackMachine.hpp dan StackMachine.cpp

Tabel 2.4.8. Implementasi kelas StackMachine

Kelas/Fungsi/Prosedur	Penjelasan
StackMachine() <pre>// inisialisasi stack ke keadaan kosong dan alokasi seluruh array StackMachine::StackMachine() : sp_(-1), bp_(0) { stack_.resize(MAX_STACK); }</pre>	Konstruktor yang menginisialisasi stack kosong
void push(StackVal val) <pre>void StackMachine::push(StackVal val) { if (sp_ >= MAX_STACK - 1) { throw std::runtime_error("Runtime Error: Stack Overflow"); } stack_[++sp_] = std::move(val); }</pre>	Mendorong nilai ke puncak stack dengan menaikkan <code>sp_</code> terlebih dahulu.
StackVal pop() <pre>StackVal StackMachine::pop() { if (sp_ < 0) { throw std::runtime_error("Runtime Error: Stack Underflow"); } return stack_[sp_--]; }</pre>	Mengambil dan mengembalikan nilai dari puncak stack, lalu menurunkan <code>sp_</code> .
StackVal top() const <pre>StackVal StackMachine::top() const { if (sp_ < 0) { throw std::runtime_error("Runtime Error: Stack Underflow"); } return stack_[sp_]; }</pre>	Membaca nilai dari puncak stack tanpa mengeluarkannya
bool empty() const <pre>bool StackMachine::empty() const { return sp_ < 0; }</pre>	Mengembalikan <code>true</code> jika stack kosong (<code>sp_ < 0</code>). Digunakan sebagai pengecekan kondisi sebelum operasi <code>pop</code> atau <code>top</code> untuk menghindari underflow.
void saveContext(int paramCount, bool hasReturnSlot, int staticLink, int returnAddr) <pre>// menyimpan konteks eksekusi sebelum masuk ke subprogram baru void StackMachine::saveContext(int paramCount, bool hasReturnSlot, int staticLink, int returnAddr) { if (paramCount < 0) { throw std::runtime_error("Runtime Error: Invalid parameter count"); } int headerSize = 3 + (hasReturnSlot ? 1 : 0); int argStart = sp_ - paramCount + 1; if (argStart < 0) { throw std::runtime_error("Runtime Error: Not enough arguments on stack"); } if (sp_ + headerSize >= MAX_STACK) { throw std::runtime_error("Runtime Error: Stack Overflow"); } for (int i = sp_; i >= argStart; --i) { stack_[i + headerSize] = std::move(stack_[i]); } int oldBP = bp_; bp_ = argStart; stack_[bp_ + 0] = static_cast<int>(staticLink); stack_[bp_ + 1] = static_cast<int>(oldBP); stack_[bp_ + 2] = static_cast<int>(returnAddr); if (hasReturnSlot) { stack_[bp_ + 3] = 0; } sp_ += headerSize; }</pre>	Menyimpan konteks eksekusi sebelum masuk ke subprogram baru. Menggeser argumen yang sudah ada di stack ke atas untuk memberi ruang bagi frame header (<i>static link</i> , <i>dynamic link</i> , <i>return address</i> , dan <i>optional return slot</i>). Mengisi slot header tersebut dan mengupdate <code>bp_</code> .

void allocFrame(int n) <pre> void StackMachine::allocFrame(int n) { if (n < 3) { throw std::runtime_error("Runtime Error: Invalid frame size (must be >= 3)"); } int targetSP = bp_ + n - 1; if (targetSP >= MAX_STACK) { throw std::runtime_error("Runtime Error: Stack Overflow"); } if (targetSP > sp_) { sp_ = targetSP; } } </pre>	Mengalokasikan sisa slot frame setelah header dengan menggeser sp_ ke bp_ + n - 1.
int restoreFrame() <pre> int StackMachine::restoreFrame() { int retAddr = toInt(stack_[bp_ + 2]); int oldBP = toInt(stack_[bp_ + 1]); sp_ = bp_ - 1; bp_ = oldBP; return retAddr; } </pre>	Memulihkan sp_ dan bp_ ke nilai frame pemanggil setelah eksekusi subprogram selesai (instruksi RET). Mengembalikan <i>return address</i> yang tersimpan di slot bp_+2.
int base(int level) const <pre> int StackMachine::base(int level) const { int b = bp_; for (int i = 0; i < level; ++i) { if (b < 0 b + 1 >= MAX_STACK) { throw std::runtime_error("Runtime Error: Invalid frame chain in base()"); } b = toInt(stack_[b + 1]); } return b; } </pre>	Menelusuri rantai dynamic link sebanyak level langkah dari bp_ saat ini untuk menemukan base pointer frame pada lexical level tertentu. Digunakan oleh load() dan store() untuk mengakses variabel di scope luar.
StackVal load(int level, int addr) <pre> StackVal StackMachine::load(int level, int addr) { int b = base(level); int idx = b + addr; if (idx < 0 idx >= MAX_STACK) { throw std::runtime_error("Runtime Error: Memory Access Out of Bounds (LOD)"); } return stack_[idx]; } </pre>	Membaca nilai dari frame pada lexical level tertentu di offset addr dari base pointer level tersebut. Menggunakan base() untuk resolusi <i>static link</i> .
void store(int level, int addr, StackVal val) <pre> void StackMachine::store(int level, int addr, StackVal val) { int b = base(level); int idx = b + addr; if (idx < 0 idx >= MAX_STACK) { throw std::runtime_error("Runtime Error: Memory Access Out of Bounds (STO)"); } stack_[idx] = std::move(val); } </pre>	Menyimpan nilai ke frame pada lexical level tertentu di offset addr. Menggunakan base() untuk resolusi <i>static link</i> .
StackVal& at(int index) <pre> StackVal& StackMachine::at(int index) { if (index < 0 index >= MAX_STACK) { throw std::runtime_error("Runtime Error: Memory Access Out of Bounds"); } return stack_[index]; } </pre> const StackVal& at(int index) const <pre> const StackVal& StackMachine::at(int index) const { if (index < 0 index >= MAX_STACK) { throw std::runtime_error("Runtime Error: Memory Access Out of Bounds"); } return stack_[index]; } </pre>	Akses langsung ke slot stack berdasarkan indeks absolut. Digunakan oleh instruksi LODI dan STOI untuk <i>indirect addressing</i> pada akses elemen array.

2.4.7. File Interpreter.hpp dan Interpreter.cpp

Tabel 2.4.9. Implementasi kelas Interpreter

Kelas/Fungsi/Prosedur	Penjelasan
-----------------------	------------

Interpreter(vector<ICInstruction> code) <pre>Interpreter::Interpreter(std::vector<ICInstruction> code) : code_(std::move(code)), pc_(0), running_(false) {}</pre>	Konstruktor yang menyimpan daftar instruksi IC, menginisialisasi StackMachine vm_, program counter pc_ = 0, dan flag running_ = false.
void run() <pre>void Interpreter::run() { running_ = true; pc_ = 0; while (running_ && pc_ >= 0 && pc_ < static_cast<int>(code_.size())) { executeInstruction(code_[pc_]); } }</pre>	Memulai siklus fetch-decode-execute: mengeset running_ = true dan pc_ = 0, lalu terus memanggil executeInstruction() selama running_ bernilai true dan pc_ berada dalam rentang kode yang valid.
void executeInstruction(const ICInstruction& instr) <pre>void Interpreter::executeInstruction(const ICInstruction& instr) { const std::string& op = instr.op; int level = instr.level; int arg = instr.arg; if (op == "LIT") { vm_.push(arg); pc++; } else if (op == "LOD") { vm_.push(vm_.load(level, arg)); pc++; } else if (op == "STO") { vm_.store(level, arg, vm_.pop()); pc++; } else if (op == "LODA") { int baseAddr = vm_.base(level); vm_.push(baseAddr + arg); pc++; } else if (op == "IXA") { int idx = toInt(vm_.pop()); int baseAddr = toInt(vm_.pop()); if (idx < level idx > arg) { throw std::runtime_error("Runtime Error: Array index out of bounds"); } vm_.push(baseAddr + (idx - level)); pc++; } else if (op == "LODI") { int addr = toInt(vm_.pop()); vm_.push(vm_.at(addr)); pc++; } else if (op == "STOI") { StackVal val = vm_.pop(); int addr = toInt(vm_.pop()); vm_.at(addr) = val; pc++; } else if (op == "INT") { if (vm_.getBP() == 0 && vm_.getSP() < 0) { vm_.saveContext(0, false, 0, (int)code_.size()); } vm_.allocFrame(arg); pc++; } else if (op == "JMP") { pc_ = arg; } else if (op == "JPC") { if (toInt(vm_.pop()) == 0) { pc_ = arg; } else { pc++; } } else if (op == "CAL") { // level >= 0 : procedure, jumlah argumen = level // level < 0 : function, jumlah argumen = -level - 1 bool hasReturnSlot = (level < 0); int paramCount = hasReturnSlot ? (-level - 1) : level; int staticLink = vm_.getBP(); vm_.saveContext(paramCount, hasReturnSlot, staticLink, pc_ + 1); pc_ = arg; } else if (op == "RET") { if (vm_.getBP() == 0) { </pre>	Mengeksekusi satu instruksi berdasarkan field op: - LIT: push nilai literal ke stack. - LOD: push nilai variabel dari memori ke stack via vm_.load(). - STO: pop nilai dari stack dan simpan ke memori via vm_.store(). - LODA: push alamat absolut variabel (untuk pengalamatan tidak langsung). - IXA: hitung alamat elemen array dari base + offset indeks, dengan validasi batas. - LODI/STOI: baca/tulis nilai menggunakan alamat absolut dari stack (indirect). - INT: alokasikan frame baru via saveContext() dan allocFrame(). - JMP: set pc_ langsung ke alamat tujuan. - JPC: pop kondisi; jika 0 (false) lompat ke tujuan, jika tidak lanjut. - CAL: simpan konteks (saveContext()), lompat ke label subprogram. - RET: kembalikan frame; jika arg=1, push return value ke stack pemanggil. - OPR: operasi aritmetika (1=NEG,2=ADD,3=SUB,4=MUL,5=DIV,6=MOD), relasional (7-12), atau I/O (13=WRT,14=WRTLN).

```

        running_ = false;
    } else {
        StackVal retVal = 0;
        if (arg == 1) retVal = vm_.load(0, 3);
        pc_ = vm_.restoreframe();
        if (arg == 1) vm_.push(retVal);
    }
} else if (op == "OPR") {
    switch (arg) {
        case 1: {
            StackVal val = vm_.pop();
            if (is_dbl(val)) {
                double res = toDouble(val);
                if (std::isinf(res)) {
                    throw std::runtime_error("Runtime Error: Real overflow");
                }
                vm_.push(res);
            } else if (std::holds_alternative<int>(val)) {
                int64_t res = -static_cast<int64_t>(toInt(val));
                if (res > std::numeric_limits<int>::max()) {
                    throw std::runtime_error("Runtime Error: Integer overflow");
                }
                if (res < std::numeric_limits<int>::min()) {
                    throw std::runtime_error("Runtime Error: Integer underflow");
                }
                vm_.push(static_cast<int>(res));
            } else {
                throw std::runtime_error("Runtime Error: Invalid operand type for Negate");
            }
            pc_++;
            break;
        }
        case 2: {
            StackVal y = vm_.pop();
            StackVal x = vm_.pop();
            if (is_str(x) || is_str(y)) {
                vm_.push(toString(x) + toString(y));
            } else if (is_dbl(x) || is_dbl(y)) {
                double res = toDouble(x) + toDouble(y);
                if (std::isinf(res)) {
                    throw std::runtime_error("Runtime Error: Real overflow");
                }
                vm_.push(res);
            } else if (std::holds_alternative<int>(x) && std::holds_alternative<int>(y)) {
                int64_t res = static_cast<int64_t>(toInt(x)) + static_cast<int64_t>(toInt(y));
                if (res > std::numeric_limits<int>::max()) {
                    throw std::runtime_error("Runtime Error: Integer overflow");
                }
                if (res < std::numeric_limits<int>::min()) {
                    throw std::runtime_error("Runtime Error: Integer underflow");
                }
                vm_.push(static_cast<int>(res));
            } else {
                throw std::runtime_error("Runtime Error: Invalid operand types for Add");
            }
            pc_++;
            break;
        }
    }
}

```

```

        case 3: {
            StackVal y = vm_.pop();
            StackVal x = vm_.pop();
            if (is_dbl(x) || is_dbl(y)) {
                double res = toDouble(x) - toDouble(y);
                if (std::isinf(res)) {
                    throw std::runtime_error("Runtime Error: Real overflow");
                }
                vm_.push(res);
            } else if (std::holds_alternative<int>(x) && std::holds_alternative<int>(y)) {
                int64_t res = static_cast<int64_t>(toInt(x)) - static_cast<int64_t>(toInt(y));
                if (res > std::numeric_limits<int>::max()) {
                    throw std::runtime_error("Runtime Error: Integer overflow");
                }
                if (res < std::numeric_limits<int>::min()) {
                    throw std::runtime_error("Runtime Error: Integer underflow");
                }
                vm_.push(static_cast<int>(res));
            } else {
                throw std::runtime_error("Runtime Error: Invalid operand types for Subtract");
            }
            pc_++;
            break;
        }
        case 4: {
            StackVal y = vm_.pop();
            StackVal x = vm_.pop();
            if (is_dbl(x) || is_dbl(y)) {
                double res = toDouble(x) * toDouble(y);
                if (std::isinf(res)) {
                    throw std::runtime_error("Runtime Error: Real overflow");
                }
                vm_.push(res);
            } else if (std::holds_alternative<int>(x) && std::holds_alternative<int>(y)) {
                int64_t res = static_cast<int64_t>(toInt(x)) * static_cast<int64_t>(toInt(y));
                if (res > std::numeric_limits<int>::max()) {
                    throw std::runtime_error("Runtime Error: Integer overflow");
                }
                if (res < std::numeric_limits<int>::min()) {
                    throw std::runtime_error("Runtime Error: Integer underflow");
                }
                vm_.push(static_cast<int>(res));
            } else {
                throw std::runtime_error("Runtime Error: Invalid operand types for Multiply");
            }
            pc_++;
            break;
        }
        case 5: {
            StackVal y = vm_.pop();
            StackVal x = vm_.pop();
            if (is_dbl(x) || is_dbl(y)) {
                double dy = toDouble(y);
                if (dy == 0.0) {
                    throw std::runtime_error("Runtime Error: Division by zero");
                }
                double res = toDouble(x) / dy;
                if (std::isinf(res)) {
                    throw std::runtime_error("Runtime Error: Real overflow");
                }
            }
        }
    }
}

```

<pre> } vm_.push(res); } else if (std::holds_alternative<int>(x) && std::holds_alternative<int>(y)) { int iy = toInt(y); if (iy == 0) { throw std::runtime_error("Runtime Error: Division by zero"); } int64_t res = static_cast<int64_t>(toInt(x)) / iy; if (res > std::numeric_limits<int>::max()) { throw std::runtime_error("Runtime Error: Integer overflow"); } if (res < std::numeric_limits<int>::min()) { throw std::runtime_error("Runtime Error: Integer underflow"); } vm_.push(static_cast<int>(res)); } else { throw std::runtime_error("Runtime Error: Invalid operand types for Divide"); } pc_++; break; } case 6: { StackVal y = vm_.pop(); StackVal x = vm_.pop(); if (std::holds_alternative<int>(x) && std::holds_alternative<int>(y)) { int iy = toInt(y); if (iy == 0) { throw std::runtime_error("Runtime Error: Division by zero"); } int64_t res = static_cast<int64_t>(toInt(x)) % iy; if (res > std::numeric_limits<int>::max()) { throw std::runtime_error("Runtime Error: Integer overflow"); } if (res < std::numeric_limits<int>::min()) { throw std::runtime_error("Runtime Error: Integer underflow"); } vm_.push(static_cast<int>(res)); } else { throw std::runtime_error("Runtime Error: Invalid operand types for Modulo"); } pc_++; break; } case 7: case 8: case 9: case 10: case 11: case 12: { StackVal y = vm_.pop(); StackVal x = vm_.pop(); if (is_str(x) is_str(y)) { vm_.push(compare(toString(x), toString(y), arg)); } else if (is_db1(x) is_db1(y)) { vm_.push(compare(toDouble(x), toDouble(y), arg)); } else if (std::holds_alternative<int>(x) && std::holds_alternative<int>(y)) { vm_.push(compare(toInt(x), toInt(y), arg)); } else { throw std::runtime_error("Runtime Error: Invalid operand types for Comparison"); } pc_++; break; } case 13: { std::cout << toString(vm_.pop()); pc_++; break; } case 14: { std::cout << toString(vm_.pop()) << std::endl; pc_++; break; } default: throw std::runtime_error("Runtime Error: Unknown OPR code: " + std::to_string(arg)); } } else { throw std::runtime_error("Runtime Error: Unknown opcode: " + op); } } </pre>	
---	--

2.4.8. File main.cpp

Tabel 2.4.10. Implementasi kelas main

Kelas/Fungsi/Prosedur	Penjelasan
int main(int argc, char* argv[])	Fungsi utama yang mengorkestrasi seluruh pipeline kompilasi dan interpretasi: 1. Membaca file sumber dari argumen argv[1] menggunakan FileUtil::readFile(). 2. Melakukan tokenisasi dengan Lexer. 3. Melakukan parsing token menjadi parse tree dengan Parser. Jika ada error, mencetak pesan dan keluar.

```

int main(int argc, char* argv[]) {
    if (argc < 2) {
        std::cerr << "Usage: ./arion-interpretor <input.txt>\n";
        return 1;
    }
    std::string src = FileUtil::readFile(argv[1]);

    Lexer lexer(src);
    std::vector<Token> tokens = lexer.tokenize();
    Parser parser(tokens);
    ParseNode* parseTree = parser.parse();

    if (parser.hasError()) {
        for (const auto& e : parser.errors())
            std::cerr << e.message << "\n";
        delete parseTree;
        return 1;
    }

    ASTBuilder builder;
    ASTNode* ast = builder.build(parseTree);

    SemanticAnalyzer sa;
    sa.analyze(ast);

    if (sa.hasError()) {
        std::cerr << "Semantic Error detected.\n";
        delete parseTree;
        return 1;
    }

    ICGenerator icg(sa.getSymbolTable());
    std::vector<ICInstruction> code = icg.generate(ast);

    for (const auto& inst : code) {
        std::cout << inst.line << " : " << inst.op << " " << inst.level << " " << inst.arg << "\n";
    }

    // Run Interpreter
    Interpreter interpreter(code);
    try {
        interpreter.run();
    } catch (const std::exception& e) {
        std::cerr << e.what() << "\n";
        delete parseTree;
        return 1;
    }

    delete parseTree;
    return 0;
}

```

4. Mengonversi parse tree menjadi AST dengan ASTBuilder.
5. Menjalankan analisis semantik dengan SemanticAnalyzer. Jika ada error semantik, keluar.
6. Menghasilkan intermediate code dengan ICGenerator::generate().
7. Mencetak seluruh instruksi IC ke stdout.
8. Menjalankan interpreter dengan Interpreter::run(). Menangkap runtime exception dan mencetak pesan error ke stderr.

2.4. Alur Kerja Program

Alur kerja program *intermediate code generator* dan *interpreter* adalah sebagai berikut:

1. Program menerima path file sumber (.txt) sebagai argumen command-line. FileUtil::readFile() membaca seluruh konten file ke dalam string.
2. Lexer memproses string sumber dan menghasilkan daftar token. Jika tokenisasi gagal, program menghentikan eksekusi dan mencetak pesan error ke stderr.
3. Parser menerima daftar token dan menghasilkan parse tree menggunakan pendekatan recursive descent. Jika terdapat kesalahan sintaksis, seluruh error dilaporkan dan program berhenti.
4. ASTBuilder melakukan traversal depth-first terhadap parse tree dan mengonversinya menjadi AST yang lebih ringkas, memetakan setiap aturan grammar ke node AST yang sesuai (ProgramNode, BlockNode, AssignNode, BinOpNode, dan sebagainya).
5. SemanticAnalyzer menganalisis AST secara rekursif menggunakan pola Visitor. Setiap node dianotasi dengan SemanticInfo yang berisi kode tipe, indeks entri di SymbolTable, dan lexical level. Semua error semantik (undeclared identifier, type mismatch, duplikasi deklarasi) dikumpulkan ke dalam vector tanpa menghentikan traversal, sehingga seluruh kesalahan dapat dilaporkan sekaligus di akhir. Jika terdapat error semantik, program berhenti.

6. ICGenerator menerima AST yang telah didekorasi dan SymbolTable dari SemanticAnalyzer, lalu melakukan traversal depth-first untuk menghasilkan daftar ICInstruction. Selama traversal, generator memelihara `currentLevel_` untuk menghitung relative lexical level dan `frameStack_` untuk membedakan frame prosedur dari fungsi. Label subprogram dicatat di `funcLabels_` dan digunakan sebagai target instruksi CAL. Instruksi JMP/JPC yang alamat tujuannya belum diketahui saat di-emit diselesaikan kemudian melalui mekanisme backpatch.
7. Seluruh instruksi IC yang dihasilkan dicetak ke stdout dalam format: nomor_baris op level arg.
8. Interpreter menerima daftar instruksi IC dan menjalankan siklus fetch-decode-execute. Pada setiap siklus, instruksi yang ditunjuk program counter (`pc_`) diambil, dieksekusi oleh `executeInstruction()`, dan `pc_` diperbarui. Eksekusi berakhir ketika instruksi RET program utama dijalankan (kondisi `bp_ == 0`) atau ketika terjadi runtime error (division by zero, stack overflow, array out of bounds). Runtime error ditangkap di `main()` dan dicetak ke stderr.

2.5. Justifikasi Implementasi

Program *intermediate code generator* dan *interpreter* ditulis dengan bahasa pemrograman C++ mengikuti milestone-milestone sebelumnya. Pendekatan *visitor* secara rekursif dipilih agar alur pembangkitan kode (program $\rightarrow$ blok $\rightarrow$ statement $\rightarrow$ ekspresi) mengikuti struktur AST secara alami dan memungkinkan pemisahan implementasi ke dalam beberapa file sesuai kategori node tanpa menambah kompleksitas pada kelas utama. Model *stack machine* dipilih karena memiliki mekanisme eksekusi yang sederhana dan tidak memerlukan pengelolaan *register* secara eksplisit, sehingga instruksi dapat dihasilkan secara langsung selama traversal AST. Instruksi lompatan menggunakan teknik *backpatching* karena alamat tujuan seringkali belum diketahui saat instruksi pertama kali dibangkitkan, sehingga alamat tersebut dapat diisi setelah target lompatan tersedia tanpa perlu melakukan traversal ulang. Pengelolaan memori *runtime* dipisahkan ke dalam kelas `StackMachine` karena tanggung jawab pengelolaan *activation record* bersifat ortogonal terhadap proses interpretasi instruksi, sehingga logika eksekusi dan manajemen memori dapat dikembangkan secara independen. Representasi nilai pada *stack* menggunakan `std::variant` untuk mendukung berbagai tipe data primitif Arion dalam satu struktur yang seragam sekaligus menjaga *type safety* pada saat *runtime*. Pengelolaan frame mengikuti konvensi *activation record* klasik yang memuat *static link*, *dynamic link*, *return address*, dan *return value*, sehingga akses variabel lintas cakupan dan pemanggilan subprogram dapat dilakukan sesuai aturan *lexical scoping*. Error pada saat *runtime*, seperti *division by zero*, *stack overflow*, dan akses memori tidak valid,

ditangani melalui mekanisme *exception* agar pelaporan kesalahan terstandarisasi dan pesan yang dihasilkan lebih informatif bagi pengguna.

BAB 3

PENGUJIAN

Tabel 3.1. Pengujian program

No	Input	Output
1	<pre> program Test; var x: integer; begin x := 5; writeln(x) end. </pre>	<pre> 0: INT 0 4 1: LIT 0 5 2: STO 0 3 3: LOD 0 3 4: OPR 0 14 5: RET 0 0 5 </pre>
2	<pre> program IfTest; var x, y: integer; begin x := 10; if x > 5 then y := x * 2 else y := x div 2; writeln(y) end. </pre>	<pre> 0: INT 0 5 1: LIT 0 10 2: STO 0 3 3: LOD 0 3 4: LIT 0 5 5: OPR 0 11 6: JPC 0 12 7: LOD 0 3 8: LIT 0 2 9: OPR 0 4 10: STO 0 4 11: JMP 0 16 12: LOD 0 3 13: LIT 0 2 14: OPR 0 5 15: STO 0 4 16: LOD 0 4 17: OPR 0 14 18: RET 0 0 20 </pre>

3	<pre> program WhileTest; var y: integer; begin y := 5; while y > 0 do begin writeln(y); y := y - 1 end end. </pre>	<pre> 0: INT 0 4 1: LIT 0 5 2: STO 0 3 3: LOD 0 3 4: LIT 0 0 5: OPR 0 11 6: JPC 0 14 7: LOD 0 3 8: OPR 0 14 9: LOD 0 3 10: LIT 0 1 11: OPR 0 3 12: STO 0 3 13: JMP 0 3 14: RET 0 0 5 4 3 2 1 </pre>
4	<pre> program Arith; var a, b, c: integer; begin a := 3; b := 4; c := (a + b) * 2; writeln(c) end. </pre>	<pre> 0: INT 0 6 1: LIT 0 3 2: STO 0 3 3: LIT 0 4 4: STO 0 4 5: LOD 0 3 6: LOD 0 4 7: OPR 0 2 8: LIT 0 2 9: OPR 0 4 10: STO 0 5 11: LOD 0 5 12: OPR 0 14 13: RET 0 0 14 </pre>

5

```
program FuncProcTest;
var
  result: integer;

function add(a: integer; b: integer): integer;
begin
  add := a + b
end;

procedure printTwice(n: integer);
var
  doubled: integer;
begin
  doubled := n * 2;
  writeln(n);
  writeln(doubled)
end;

begin
  result := add(3, 4);
  printTwice(result)
end.
```

```
0: INT 0 4
1: JMP 0 18
2: INT 2 6
3: LOD 0 4
4: LOD 0 5
5: OPR 0 2
6: STO 0 3
7: RET 0 1
8: INT 1 5
9: LOD 0 3
10: LIT 0 2
11: OPR 0 4
12: STO 0 4
13: LOD 0 3
14: OPR 0 14
15: LOD 0 4
16: OPR 0 14
17: RET 0 0
18: LIT 0 3
19: LIT 0 4
20: CAL -3 2
21: STO 0 3
22: LOD 0 3
23: CAL 1 8
24: RET 0 0
7
14
```

6	<pre> program ArrayBoundsTest; var arr: array [1 .. 5] of integer; idx: integer; begin idx := 3; arr[idx] := 42; writeln(arr[idx]); idx := 6; arr[idx] := 99 end. </pre>	<pre> 0: INT 0 9 1: LIT 0 3 2: STO 0 8 3: LODA 0 3 4: LOD 0 8 5: IXA 1 5 6: LIT 0 42 7: STOI 0 0 8: LODA 0 3 9: LOD 0 8 10: IXA 1 5 11: LODI 0 0 12: OPR 0 14 13: LIT 0 6 14: STO 0 8 15: LODA 0 3 16: LOD 0 8 17: IXA 1 5 18: LIT 0 99 19: STOI 0 0 20: RET 0 0 42 Runtime Error: Array index out of bounds </pre>
7	<pre> program OverflowTest; var a: integer; b: integer; res: integer; begin a := 2147483647; b := 1; res := a + b; writeln(res) end. </pre>	<pre> 0: INT 0 6 1: LIT 0 2147483647 2: STO 0 3 3: LIT 0 1 4: STO 0 4 5: LOD 0 3 6: LOD 0 4 7: OPR 0 2 8: STO 0 5 9: LOD 0 5 10: OPR 0 14 11: RET 0 0 Runtime Error: Integer overflow </pre>

```
program LoopsTest;
var
  i: integer;
begin
  { 1. Test While Loop }
  writeln(1111);
  i := 1;
  while i <= 3 do
  begin
    writeln(i);
    i := i + 1
  end;

  { 2. Test For-To Loop }
  writeln(2222);
  for i := 1 to 3 do
  |  writeln(i);

  { 3. Test For-Downto Loop }
  writeln(3333);
  for i := 3 downto 1 do
  |  writeln(i);

  { 4. Test Repeat-Until Loop }
  writeln(4444);
  i := 1;
  repeat
  |  writeln(i);
  |  i := i + 1
  until i == 4
end.
```

```
0: INT 0 4
1: LIT 0 1111
2: OPR 0 14
3: LIT 0 1
4: STO 0 3
5: LOD 0 3
6: LIT 0 3
7: OPR 0 12
8: JPC 0 16
9: LOD 0 3
10: OPR 0 14
11: LOD 0 3
12: LIT 0 1
13: OPR 0 2
14: STO 0 3
15: JMP 0 5
16: LIT 0 2222
17: OPR 0 14
18: LIT 0 1
19: STO 0 3
20: LOD 0 3
21: LIT 0 3
22: OPR 0 12
23: JPC 0 31
24: LOD 0 3
25: OPR 0 14
26: LOD 0 3
27: LIT 0 1
28: OPR 0 2
29: STO 0 3
30: JMP 0 20
31: LIT 0 3333
32: OPR 0 14
33: LIT 0 3
34: STO 0 3
35: LOD 0 3
36: LIT 0 1
37: OPR 0 10
38: JPC 0 46
39: LOD 0 3
40: OPR 0 14
41: LOD 0 3
42: LIT 0 1
```

		43: OPR 0 3 44: STO 0 3 45: JMP 0 35 46: LIT 0 4444 47: OPR 0 14 48: LIT 0 1 49: STO 0 3 50: LOD 0 3 51: OPR 0 14 52: LOD 0 3 53: LIT 0 1 54: OPR 0 2 55: STO 0 3 56: LOD 0 3 57: LIT 0 4 58: OPR 0 7 59: JPC 0 50 60: RET 0 0 1111 1 2 3 2222 1 2 3 3333 3 2 1 4444 1 2 3
--	--	---

BAB 4

KESIMPULAN DAN SARAN

4.1. Kesimpulan

Intermediate code generator dan *interpreter* yang dikembangkan telah berhasil melengkapi tahapan kompilasi dengan menyediakan mekanisme translasi dan eksekusi program setelah proses analisis semantik selesai dilakukan. *Generator intermediate code* mengadopsi pola *visitor* untuk melakukan traversal *Abstract Syntax Tree* (AST) secara *depth-first* dan menghasilkan representasi *intermediate code* berbasis *stack machine* yang dapat digunakan sebagai perantara antara hasil analisis program dan proses eksekusi. Integrasi informasi simbol yang dihasilkan pada tahap analisis semantik memungkinkan proses pembangkitan instruksi dilakukan secara konsisten terhadap aturan *lexical scoping* yang digunakan oleh bahasa Arion. Representasi *instruction set* yang digunakan mampu merepresentasikan berbagai konstruksi bahasa, seperti ekspresi, assignment, percabangan, perulangan, serta pemanggilan prosedur dan fungsi, ke dalam bentuk instruksi yang lebih sederhana dan mudah dieksekusi. *Interpreter* yang dikembangkan mampu mengeksekusi *intermediate code* melalui mekanisme *virtual machine* berbasis *stack machine* dengan dukungan *activation record*, *static link*, *dynamic link*, dan pengelolaan memori *runtime*. Sistem juga mampu menangani berbagai kondisi kesalahan saat eksekusi, seperti *division by zero*, *stack overflow*, dan *array out of bounds*.

4.2. Saran

Pengembangan komponen *intermediate code generator* dan *interpreter* membutuhkan spesifikasi bahasa yang jelas dan konsisten agar seluruh konstruksi bahasa dapat diterjemahkan dan dieksekusi secara tepat. Dengan spesifikasi yang lebih rinci, proses perancangan *instruction set*, mekanisme *runtime*, dan penanganan kasus-kasus khusus dapat dilakukan dengan lebih terarah dan mengurangi kemungkinan perbedaan interpretasi selama implementasi. Pada pengembangan selanjutnya, sistem dapat diperluas dengan dukungan fitur bahasa yang lebih lengkap serta mekanisme optimasi *intermediate code* untuk meningkatkan efisiensi eksekusi program. Selain itu, penyediaan dokumentasi yang lebih rinci mengenai perilaku *runtime* dan semantik bahasa dapat membantu proses pengembangan, pengujian, dan pemeliharaan sistem secara keseluruhan.

BAB 5

LAMPIRAN

5.1.Link Release Repository GitHub

5.2.Pembagian Tugas

Tabel 5.3. Pembagian tugas

NIM	Pembagian tugas	Persentase
13524001	1. Implementasi ICGen_Expressions 2. Laporan	20%
13524027	1. Implementasi Interpreter 2. Laporan	20%
13524029	1. Implementasi ICGen_Statements dan ICGen_Subprograms 2. Laporan	20%
13524089	1. Implementasi stack machine 2. Laporan	20%
13524093	1. Implementasi ICInstruction dan ICGenerator 2. Laporan	20%

5.3.Referensi

Aho, Alfred V, et al. *Compilers : Principles, Techniques, and Tools*. Noida, India, Dorling Kindersley (India) Pvt. Ltd., Licensees Of Pearson Education In South Asia, 2014.

Spivak, Ruslan. "Let's Build A Simple Interpreter. Part 13." *Ruslan's Blog*, 2015,
<https://ruslanspivak.com/lbasi-part13/>

GeeksforGeeks. "Introduction to Intermediate Representation (IR)." GeeksforGeeks, 2025,
<https://www.geeksforgeeks.org/compiler-design/introduction-to-intermediate-representation/>